

Week 3 — Public Key Cryptography, Digital Signatures, Forks, 51% Attack, and Evolution of Money

1) Public key cryptography — the basic idea

The lecture begins with **public key cryptography**. It says each user generates a **public-private key pair**. Public keys are “very public,” and private keys are “very private.” It also shows the idea that Alice can lock data using her private key so that her public key can unlock it, and optionally she can also use Bob’s public key so that only Bob’s private key can unlock it.

This is a very important concept, so let’s slow it down.

1.1 What is a key pair?

In public key cryptography, every user has **two keys**:

- a **public key**
- a **private key**

These two keys are mathematically related, but they do not play the same role.

The **public key** can be shared openly.

The **private key** must be kept secret.

That is why the slide says:

- public keys: very public
- private keys: very private.

1.2 Why do we need two keys?

Because blockchain needs two different things:

- the ability to prove who authorized a transaction
- sometimes the ability to ensure only the intended receiver can access or spend something

A single shared-secret system is not enough for this kind of public distributed network.

1.3 The lecture's "locking" language

The slide uses the word **lock**. That is a very good way to think about it.

If Alice uses a key to "lock" data, then the corresponding matching key is needed to unlock it.

So the lecture is trying to teach you that blockchain transactions are tied to ownership and authorization through keys, not through just usernames or passwords.

2) Alice using her private key and Bob's public key

The lecture says Alice can do two things:

1. use **her private key**, so only her **public key** can unlock
2. optionally also use **Bob's public key**, so only Bob's **private key** can unlock.

This sounds confusing at first, so let's explain what she is trying to convey.

2.1 Using Alice's private key

When Alice uses her **private key**, the main purpose is **authentication**.

It means:

- the data is tied to Alice
- others can use Alice's **public key** to check that it was really authorized by her

This is the basis of **digital signatures**.

2.2 Using Bob's public key

If Alice also uses **Bob's public key**, then only Bob's **private key** can unlock/decrypt that part.

This gives **receiver-specific protection**.

That means:

- not only can Alice prove she sent it,
- but she can also make it so only Bob can access the protected content

2.3 Why is the second part optional?

Because sometimes in blockchain, the main need is not secrecy of the transaction text itself. Often the key need is:

- proving who sent it
- proving it was not forged

So sender authentication may be enough in some contexts.

But if confidentiality is also needed, then the receiver's public key can also be used.

This is why the slide says “optionally.”

3) Bob receives some data — can he be sure it came from Alice?

This thought question appears several times in the lecture.

This is a super important question.

Because blockchain is not just about “can Bob read the message?”

It is also about:

- can Bob trust the sender?
- can Bob trust the message content?
- can Bob trust that nothing was altered?

The entire topic of digital signatures is really answering this question.

So always remember:

receiving data is not the same as knowing who truly sent it.

That is why authentication matters.

4) Digital signatures — the simple-minded approach

The lecture then explains a very basic version of digital signatures:

- Alice “signs” with her private key
- signing is treated like encrypting/encoding
- Alice sends Bob the original document plus the signature
- Bob uses Alice's public key to decode the signature and compare
- if comparison works, it seems to verify:
 - Alice sent it
 - the message was not forged

- but anyone else can also decode it
- and if we do not want that, then we need more thought.

Let's understand this carefully.

4.1 What is the basic idea of signing?

Alice has a message.

She uses her **private key** to create a signature on that message.

Then she sends:

- the message
- the signature

Bob receives both.

He uses **Alice's public key** to verify the signature.

If verification succeeds, it suggests two things:

- the message is tied to Alice's private key
- the message has not been altered

4.2 What properties does this seem to provide?

The lecture names two major properties:

Authentication / non-repudiation

If only Alice has her private key, then a valid signature suggests Alice really authorized it.

Non-repudiation means Alice cannot later easily deny making that signed transaction.

Integrity

If the message changed after signing, verification would fail.

So the message appears unmodified.

4.3 What is the limitation?

The lecture says anyone else can also decode the message, and maybe that is okay — but what if we do not want that?

This tells you that digital signatures and confidentiality are different goals.

A signature proves authorship and integrity.
It does **not automatically guarantee secrecy**.

This is a very important distinction.

5) PKI and transactions on Bitcoin blockchain

The lecture then directly connects public key cryptography with Bitcoin transactions.

It says:

- cryptocurrency owned by Alice came from a previous transaction
- that value was previously locked to Alice's **public key**
- Alice unlocks the value using her **private key**
- then she makes a new transaction
- she locks some value to Bob's **public key**
- only Bob can later unlock using his **private key**
- she also locks the remaining value back to **her own public key**
- and she cannot later deny making the transaction.

This is one of the most important Week 3 concepts.

5.1 Value is locked to a public key

This is how you should think about ownership in Bitcoin-style systems.

The value is not just “inside Alice’s account” in the casual banking sense.

Instead, previous transactions created value that is spendable only by someone who can prove control of the right private key.

So if value is locked to Alice’s public key, that means only Alice’s corresponding private key can authorize spending it.

5.2 Alice unlocks using her private key

This is the spending authorization step.

By using her private key, Alice proves:
“I am the rightful controller of this value.”

That is why no one else can spend her coins unless they have her private key.

5.3 Alice locks value to Bob's public key

Now the transaction creates the next ownership condition.

Instead of value remaining with Alice, some of it is newly locked to Bob's public key.

This means Bob becomes the next rightful spender, because only Bob's private key can later unlock it.

5.4 Alice locks the remaining value back to herself

This is a very important detail students often miss.

Suppose Alice has access to some earlier locked value worth more than what she wants to pay Bob.

She does not just partially unlock the exact amount and leave the rest floating around.

The transaction typically:

- unlocks the full old value
- sends some to Bob
- sends the remainder back to Alice under a new locking condition

So the leftover is also explicitly handled.

5.5 Why can't Alice deny the transaction later?

Because she used her **private key** to authorize it.

That is the non-repudiation idea:
the signed transaction is cryptographic evidence of her authorization.

6) Digital signatures: integrity + authentication

The next slide formalizes the goal:
check message integrity and sender authenticity.

It shows:

- message M
- Alice's private key
- signature Y
- transfer of Y and M

- Bob uses Alice's public key
- if the decoded result matches M, then authenticity and integrity seem verified.

Let's explain these two properties very clearly.

6.1 Integrity

Integrity means:

the message was not changed.

If Alice signs message M, and Bob verifies against M, then any change in the message should break the match.

So integrity is about content staying unaltered.

6.2 Authentication

Authentication means:

the sender is really who they claim to be.

If the signature verifies under Alice's public key, that suggests the corresponding private key was used.

Since Alice should be the only one with that private key, the message appears to come from her.

6.3 Why these two are both necessary

You do not just want a message that is intact; you also want to know it came from the right person.

And you do not just want proof of sender; you also want proof the message content was not changed.

So blockchain transactions need both:

- **who sent it**
- **what exactly was sent**

That is why digital signatures are so central.

7) Kevin can claim to be Alice — the problem with the naive approach

This is one of the smartest parts of the lecture.

The slide shows that **Kevin**, even without Alice's private key, can still cause trouble. The slide says:

- Kevin can claim to be Alice
- Bob may think the message is legit
- Kevin can control one text but not the other
- bad news: it is still an injection of garbage data
- solution: **try hashing before signing**.

This needs careful explanation.

7.1 What is the problem?

The lecture is saying:

if you think too simplistically about "signing the whole raw message," then there is still room for trouble.

An attacker might create misleading message/signature pair behavior that confuses the receiver or injects meaningless junk.

The slide's core point is not to make you memorize the exact Kevin diagram mechanically. The real point is:

Naive direct signing is not the safest conceptual design.

7.2 Good news and bad news

The slide says Kevin can control one text but not the other.

This means he cannot truly forge Alice's exact private-key-based authorization in the proper sense.

That is the good news.

But he can still inject garbage or abuse the simplistic verification approach enough to create confusion or bad data flow.

That is the bad news.

7.3 Why this matters

The lecture wants you to understand that cryptographic protocols need careful design.

Just having public and private keys is not enough.
You must use them in the right procedure.

That is why the lecture next moves to:
hash before signing.

8) Why we hash before signing

The next slide gives the correct approach:

1. Create a message digest using cryptographic hashing
2. Encrypt/sign the message digest with your private key
3. Send the signature along with the message

At the receiver:

- decrypt the hash value using the public key
- compute the message hash independently
- compare the two

This gives:

- sender authentication
- message integrity

And Kevin or anyone else cannot insert garbage because the hash function is one-way.

This is one of the biggest Week 3 topics.

8.1 What is a message digest?

A message digest is just the **hash** of the message.

So instead of signing the raw message directly in the simplistic picture, we:

- hash the message first
- then sign the hash

8.2 Why is this better?

Because the hash is a compact fixed-size fingerprint of the message.

If the message changes even slightly, the digest changes drastically.

So the receiver can detect any modification by comparing hash results.

8.3 The sender side

Alice:

- takes message M
- computes hash $H(M)$
- signs that hash using her private key
- sends both the message and the signature

8.4 The receiver side

Bob:

- receives the message and signature
- uses Alice's public key to recover/check the signed hash
- independently computes the hash of the received message
- compares the two hash values

If they match:

- the message appears unaltered
- the signature appears valid
- the sender appears authentic

8.5 Why Kevin can't inject garbage now

Because the hash function is one-way and very sensitive to change.

If Kevin changes the message, the receiver's newly computed hash will no longer match the signed digest.

So this method protects against message tampering and garbage injection much more robustly.

8.6 The two most important outcomes

The lecture explicitly names them:

- **Sender authentication**
- **Message integrity**

That is the clean exam answer too.

9) With all this security, can we still break a blockchain?

Now the lecture changes direction.

It asks:

with all this security, authentication, verification, etc., can we still break a blockchain?

Before answering, it says we need to understand **forks**. It explains that two miners can almost simultaneously create valid candidate blocks, both may be broadcast, both may be added to journals, and this causes a **natural/accidental fork**. Over time one becomes part of the ledger through the longest-chain rule. These natural forks are frequent and automatically resolved.

This is very important.

9.1 What is a natural fork?

A natural fork happens when two valid blocks are found at nearly the same time.

Different nodes may receive different ones first.

So temporarily, not everyone agrees on the same latest block.

This creates two branches.

9.2 Is that automatically a disaster?

No.

The lecture says these accidental forks are frequent and automatically resolved.

That means natural forks are part of normal blockchain behavior, not necessarily an attack.

9.3 How are they resolved?

Eventually one branch becomes longer / accumulates more accepted work.

Then the network converges on that branch.

The other branch becomes stale/orphaned.

So when people casually say “fork,” it is important to distinguish:

- accidental temporary fork
- deliberate purposeful fork

The lecture says when we talk of forks in a stronger sense, we usually do **not** mean the accidental kind.

10) Forks with a purpose: Hard fork

The lecture defines a **hard fork** as something that:

- loosens the rules for newly created blocks
- examples: increase mining reward, increase max block size
- a block invalid under old rules may become valid under new rules
- only upgraded nodes accept new blocks
- all nodes must upgrade to avoid a split, and often they do not.

This is highly testable.

10.1 What does “loosens the rules” mean?

It means the new rules are more permissive.

Example:

if old rules said maximum block size is small, and new rules allow a larger one, then some blocks rejected before may now be accepted.

So new rules are broader or more liberal.

10.2 Why does this cause trouble with old nodes?

Because old nodes still follow the old stricter/older acceptance logic.

So when upgraded nodes create blocks under the new looser rules, old nodes may reject them.

That means the network can split into two groups:

- upgraded group accepting the new blocks
- non-upgraded group rejecting them

10.3 Why must everyone upgrade?

Because otherwise the old and new groups may permanently disagree on the valid chain.

That is what causes a chain split.

So a hard fork is more disruptive.

10.4 One-line memory trick

Hard fork = looser new rules, old nodes reject, split likely if nodes don't all upgrade.

11) Forks with a purpose: Soft fork

The lecture defines a **soft fork** as something that:

- tightens the rules for newly created blocks
- examples: decrease mining reward, decrease max block size
- a block valid under old rules may be invalid under new rules
- all nodes, upgraded or not, will accept the new blocks
- if majority upgraded, old nodes do not need to upgrade
- blockchain is unlikely to split.

This is the opposite style of rule change.

11.1 What does “tightens the rules” mean?

It means the new rules are stricter.

So some blocks that would have been okay before are no longer okay.

11.2 Why is split less likely?

Because blocks that satisfy the stricter new rules also satisfy the old broader rules.

So even non-upgraded nodes can still accept them.

That is the key compatibility advantage.

11.3 What happens to non-compliant blocks?

If upgraded majority nodes reject blocks that violate the new stricter rules, those blocks tend to become stale.

The slide even labels such a block as becoming stale by upgraded majority nodes.

11.4 One-line memory trick

Soft fork = stricter new rules, backward-compatible acceptance, split less likely.

12) Hard fork vs soft fork — the real difference

Let's make this crystal clear.

Hard fork

- loosens rules
- new-valid blocks may be old-invalid
- old nodes reject them
- everyone must upgrade to stay unified
- split risk is high

Soft fork

- tightens rules
- new-valid blocks are still acceptable to old nodes
- old nodes can continue following the chain
- majority upgrade is often enough
- split risk is lower

This difference is extremely likely to be asked in short-answer form.

13) The 51% attack — basic setup

Lecture 06 introduces the **51% attack**.

The first slides show:

- truthful miners adding blocks to the public chain by broadcasting them
- malicious miner adding blocks to a **private chain** without broadcasting them.

This is one of the biggest security concepts in blockchain.

13.1 What is the core idea?

The attacker secretly builds an alternative private branch while honest miners keep building the public chain.

The malicious miner does **not** immediately reveal the private branch.

Instead, they keep extending it in secret.

13.2 Why is this dangerous?

Because if the attacker eventually makes the private chain longer than the public one, then according to the protocol, the network may accept the attacker's chain as the winning chain.

That is the danger.

14) Step-by-step 51% attack story

The lecture explains it over several slides, so let's tell it as one story.

Step 1: Honest public chain continues

Truthful miners keep broadcasting blocks and extending the public chain normally.

Step 2: Malicious miner secretly builds a private chain

Instead of publishing their mined blocks, the attacker keeps them private.

So now there are effectively two branches:

- public truthful branch
- secret attacker branch

Step 3: Malicious miner spends BTC on the public chain

The lecture explicitly says the malicious miner spends BTC for buying items and records that spending on the truthful public chain.

So from the merchant's perspective, the payment appears to have happened.

Step 4: But the attacker excludes that spend from the private chain

The lecture says the attacker excludes those transactions from the private chain, so in that private history they still own the BTC.

This is the key trick.

Publicly: they spent the coins.

Privately: they pretend they never spent them.

Step 5: Attacker mines faster due to higher hashing power

The lecture says the malicious miner adds blocks to the private chain faster owing to higher compute power / higher hashing power.

That is why this is called a **51% attack**:
the attacker effectively has majority hashing power.

Step 6: Attacker broadcasts the private chain once it is longer

The lecture says the malicious miner broadcasts the chain once it is longer than the public one.

Step 7: Honest miners must accept the longest chain

Because the protocol says to follow the longest valid chain, the honest network switches to the attacker's chain.

Step 8: Old public chain is abandoned

That means the earlier public branch — including the attacker's spend transaction — gets abandoned.

Step 9: Double spending happens

Now the attacker has:

- already spent coins in the public version
- but still appears to own them in the adopted private-longer chain

So they can spend them again.

This is the double-spending attack scenario.

15) Why is it called a 51% attack?

Because to reliably outrun the honest public chain, the attacker typically needs **majority hashing power**.

If the honest network has more total mining power than the attacker, then over time the honest chain is more likely to stay ahead.

But if the attacker controls more than half, they can statistically outpace the public chain often enough to make this attack practical.

So the 51% attack is really about **majority control over block production power** in PoW.

16) What exactly is the attacker achieving?

The attacker is **not** magically breaking hash functions or stealing everyone's keys.

Instead, the attacker is abusing the longest-chain rule using superior mining power.

So the attack is more about **reorganizing history** than about cracking cryptography directly.

That is a very important distinction.

17) What happens to the transaction that was on the abandoned chain?

The lecture itself asks:

"What happens to the transaction in which the malicious miner had spent? Does it go back to the mempool?"

Your sir may definitely ask this.

Conceptual answer

If the old public chain is abandoned, transactions that existed only on that abandoned branch are no longer part of the accepted main chain.

Then one of two things can happen conceptually:

- if still valid and not conflicting, they may return to mempool / be reconsidered
- but in the malicious attacker's spending case, the conflicting history may mean it does not survive as a valid accepted transaction on the winning chain

For your course level, the safest answer is:

That earlier public-chain spend is no longer part of the accepted ledger once the chain is abandoned, which is exactly why the attacker can effectively double-spend.

18) Can blockchain still be broken despite signatures and hashing?

Yes — and this lecture shows how.

Even if:

- signatures authenticate transactions,
- hashes chain blocks,
- and verification works,

the blockchain can still be attacked at the **consensus level** if one entity controls too much mining power.

So Week 3 teaches an important security lesson:

Cryptographic security of transactions is not the same as security of the consensus process.

That is a very strong sentence to remember.

Part 2 — Evolution of Money

Now the lecture shifts away from blockchain security and starts building the background for cryptocurrencies by explaining how money evolved.

This is a very interesting topic and definitely testable in theory questions.

19) Before money — the barter system

The lecture says:

- before money
- barter system
- can make it work 3-way.

19.1 What is barter?

Barter means direct exchange of goods or services without a standard money medium.

Example:

- I give you wheat

- you give me milk

No money is involved.

19.2 Why is barter inconvenient?

Because it requires a matching of wants.

If I have goats and want rice, I must find someone who:

- has rice
- and wants goats

That is hard.

19.3 What does “can make it work 3-way” mean?

It means even if direct barter does not work between two people, it may sometimes be arranged indirectly through a third person.

Example:

- A wants B's item
- B wants C's item
- C wants A's item

A three-way exchange can solve the mismatch.

But this is still inefficient.

19.4 Why is barter historically important?

Because it shows the problem money was created to solve:
efficient exchange without requiring direct matching of wants every time.

20) Commodity money

The lecture then explains **commodity money**.

It says commodity money should have:

- divisibility
- stable value
- scarcity is the key

- it works peer-to-peer
- but it is cumbersome for large purchases
- example question: how many silver coins are needed to buy an expensive item?

20.1 What is commodity money?

Commodity money is money made from something that has **intrinsic value** of its own.

Examples may include:

- gold
- silver
- salt
- shells
- precious stones
- other goods people value directly.

20.2 What does intrinsic value mean?

It means the item is valuable even aside from its use as money.

Gold is not valuable only because we agree to call it money; it is also valued as a real scarce commodity.

20.3 Why are divisibility and stable value important?

Money should be easy to divide into smaller units and should hold reasonably stable value over time.

If it cannot be divided well or fluctuates wildly in usefulness, it is a poor money candidate.

20.4 Why is scarcity the key?

If something is too common and easily produced, it cannot function well as a store of value.

Scarcity helps preserve worth.

This is one reason gold historically worked better than very abundant materials.

20.5 Why does commodity money work peer-to-peer?

Because if both people recognize the commodity's value, they can directly exchange it without needing a bank in the middle.

So commodity money is naturally peer-to-peer.

20.6 What is the problem then?

It is cumbersome, especially for large or awkward purchases.

Imagine needing huge amounts of silver coins for something expensive.

So while commodity money solves the barter matching problem, it introduces practical transport and scaling issues.

21) Representative money

The lecture then explains **representative money**.

It says:

- government-produced money
- backed by some precious commodity
- has little to no intrinsic value of its own
- but can be exchanged for something that does have intrinsic value
- easier to transact
- enforced unforgeability
- value based on the backing instrument
- often backed by gold or silver
- and not just “some common value” people casually agree on.

This is very important.

21.1 What is representative money?

Representative money is money that stands for some underlying valuable commodity.

The note itself may not be intrinsically valuable, but it represents a claim on something valuable like gold or silver.

21.2 Why is it better than commodity money operationally?

Because paper or officially issued tokens are much easier to carry and transact with than bulky physical commodities.

So representative money improves convenience.

21.3 What does “backed” mean?

It means the currency can, in principle, be redeemed or justified based on an underlying reserve commodity.

Its value comes from that backing.

21.4 Why does the lecture say “not just some common value everyone agrees upon”?

Because representative money's value is not purely social belief alone; it is tied to a specific backing asset.

So its value is anchored in something external and tangible.

That is different from pure fiat money, which comes next.

22) US gold reserves and Fort Knox

The lecture gives examples:

- Fort Knox, Kentucky: 4583 metric tons, about 56% of US gold reserves
- Federal Reserve Bank building in New York vault: over 6000 metric tons, more than 95% belonging to other countries
- total reserves valued at about \$541B at \$118M per metric ton
- about 1/8th of NVIDIA's market cap as of Sept 16, 2025
- over the last 10 years, gold price increased slightly less than 3x while NVDA stock increased slightly more than 300x.

Why is sir showing this?

This is not random trivia.

He is trying to make representative-money backing feel real.

If currency is backed by gold, then actual physical reserves matter.

Fort Knox becomes a symbol of the old reserve-based system.

What is the conceptual lesson?

The value basis of representative money is tied to the reserve commodity, not just to collective belief alone.

That is why physical reserve discussion matters.

23) Fiat money

Finally, the lecture explains **fiat money**.

It says fiat money is:

- government-produced money like USD, Euro, PKR
- has little to no intrinsic value
- may not be backed by any precious commodity
- value is based on what the government sets
- money the government guarantees as legal tender
- and in 1971 the US severed ties with the Gold Standard.

This is very important because cryptocurrencies are often discussed in contrast with fiat.

23.1 What is fiat money?

Fiat money is currency that has value because the government declares it legal tender and the economic/legal system recognizes it.

It does not need to be redeemable for gold or silver.

23.2 Does fiat have intrinsic value?

According to the lecture: little to none.

A paper note is not valuable mainly because of the paper material itself.

Its value comes from legal, institutional, and social acceptance.

23.3 What does legal tender mean?

It means the currency is officially recognized by the government for settling debts and payments.

So the government-backed monetary system gives it authority and enforceability.

23.4 Why is 1971 important?

Because the lecture says the US severed ties with the Gold Standard in 1971.

That means the system moved more fully into fiat logic rather than gold-backed representative-money logic.

This is a major historical turning point.

Week 3 — Full connected story

Now let me connect the whole week as one big story.

Blockchain transactions need secure authorization, so the lecture begins with public key cryptography. Every user has a public-private key pair. The private key stays secret; the public key is shared. Alice can authorize data using her private key, and optionally she can also use Bob's public key when confidentiality for Bob matters. This raises the question: if Bob receives some data, can he really know it came from Alice?

That leads to digital signatures. In the naive picture, Alice signs using her private key, sends the original message plus the signature, and Bob checks it using Alice's public key. This seems to provide sender authentication and message integrity, but the lecture shows that the simplistic approach is still vulnerable to confusion or garbage injection. So the proper method is to hash the message first, then sign the digest. The receiver verifies by comparing the signed hash with the freshly computed hash. This gives two crucial properties: authentication and integrity.

The lecture then connects this to Bitcoin. Value is previously locked to Alice's public key. Alice unlocks it using her private key, pays Bob by locking some value to Bob's public key, and locks the remainder back to herself. Because she authorized the transaction with her private key, she cannot later deny making it.

After that, the lecture returns to blockchain security at the chain level. Even if transactions are cryptographically secure, blockchains can still face chain-level phenomena like forks. Natural forks happen when two miners almost simultaneously create valid blocks. Those forks are common and normally resolved automatically by the longest-chain rule. But forks can also be intentional. A hard fork loosens rules and requires widespread upgrading or the chain may split. A soft fork tightens rules and is more backward-compatible, so split is less likely.

Then comes the 51% attack. A malicious miner secretly builds a private chain while honest miners keep extending the public one. The attacker spends coins publicly but excludes that spend from the private branch. If the attacker has enough hashing power to make the private chain longer, the network follows the longest chain and abandons the public branch. This lets the attacker effectively double-spend. The lesson is that secure signatures and hashes do not alone guarantee secure consensus if one party gains majority mining power.

Finally, the lecture steps back and explains the evolution of money. First came barter, which is direct exchange but inefficient because matching wants is hard. Commodity money solved some of that because scarce intrinsically valuable things like gold or silver could serve as money directly, though large transactions were cumbersome. Representative money then improved

convenience by using government-issued money backed by precious commodities. Later, fiat money emerged, where currency has little intrinsic value and may not be backed by gold at all; instead, its value comes from government authority and legal tender status. The 1971 break from the Gold Standard marks that transition clearly.

Most important Week 3 topics to memorize

These are the highest-priority ones:

- public key vs private key
 - why Bob cannot just assume received data came from Alice
 - naive digital signatures
 - hash before signing
 - authentication vs integrity
 - value locked to public keys in Bitcoin
 - unlocking with private key
 - hard fork vs soft fork
 - natural/accidental fork vs purposeful fork
 - 51% attack step by step
 - double spending through private longer chain
 - barter, commodity money, representative money, fiat money
 - intrinsic value vs no intrinsic value
 - 1971 and the end of the US Gold Standard link
-

Ultra-short ratta revision for Week 3

Every user in public key cryptography has a public-private key pair. The private key is secret and is used for authorization/signing, while the public key is shared and used for verification. In Bitcoin-style systems, value is locked to public keys and can only be unlocked by the matching private key. Alice spends by unlocking previous value with her private key and relocking value to Bob's public key, with change relocked to herself. Naive signing of raw messages is not enough, so the proper method is to hash the message first and then sign the digest. The receiver verifies by comparing the decrypted signed hash with the newly computed hash, giving sender authentication and message integrity.

A natural fork happens when two miners find valid blocks almost simultaneously; one branch later wins by the longest-chain rule. A hard fork loosens rules and can split the chain unless everyone upgrades. A soft fork tightens rules and is more backward-compatible, so split is less

likely. In a 51% attack, a malicious miner privately builds a longer chain, publicly spends coins, excludes that spend from the private branch, then releases the longer chain so the network accepts it and the attacker can effectively double-spend.

The evolution of money goes from barter to commodity money to representative money to fiat money. Commodity money has intrinsic value and works peer-to-peer but is cumbersome. Representative money is backed by valuable reserves like gold or silver. Fiat money has little intrinsic value and may not be backed by a commodity; its value comes from government authority and legal tender status.